

Final Passdown Frontend

Final Passdown Frontend is a Vite + React application for managing digital legacy, emergency information, subscriptions, documents, contacts, memories, and end-of-life planning workflows. The app has two major surfaces:

- **User portal** for account owners and contact roles such as Legacy Contact, Guardian Contact, and Emergency Contact.
- **Admin portal** for platform management, users, plans, storage subscriptions, payment logs, queries, and notifications.

The project is designed to be served from the `/final-pass-down-frontend` base path by default.

Table of Contents

- [Tech Stack](#)
- [Project Flow](#)
- [Getting Started](#)
- [Environment Variables](#)
- [Available Scripts](#)
- [Application Architecture](#)
- [Routing Flow](#)
- [Authentication And Authorization](#)
- [API Flow](#)
- [State Management](#)
- [Main Feature Areas](#)
- [Folder Structure](#)
- [Development Notes](#)

Tech Stack

- **React 18** for the frontend UI.
- **Vite 4** for local development and production builds.
- **React Router DOM 6** for route definitions and route guards.
- **Redux + redux-persist** for persisted auth/session state.

- **Axios** for API requests.
- **Bootstrap / React Bootstrap** for layout and UI patterns.
- **Formik + Yup** for forms and validation in many screens.
- **Stripe** for subscription/payment flows.
- **Socket.IO Client** for real-time features where used.
- **Leaflet / React Leaflet and Google Maps** for map/location related UI.

Project Flow

At runtime, the application starts from `index.html`, mounts React from `src/main.jsx`, and then renders the route tree through `src/App.jsx` and `src/Routers/index.jsx`.

```
index.html
-> src/main.jsx
  -> Redux Provider
  -> redux-persist PersistGate
  -> BrowserRouter
  -> UploadProgressProvider
  -> ApiConfig
  -> App
    -> Routers
```

The routing layer separates the app into:

1. Admin guest flow

- `/admin`
- `/admin/login`
- `/admin/forget-password`
- `/admin/forget-password2`
- `/admin/forget-password3`

2. Admin protected flow

- `/admin/dashboard`
- `/admin/profile`
- `/admin/user-management`
- `/admin/subscription-management`
- `/admin/storage-subscription-management`
- `/admin/storage-subscription-logs`
- `/admin/query-management`
- `/admin/payment-logs`
- `/admin/notifications`

3. Public user flow

- `/`
- `/contact-us`
- `/privacy-policy`

- /terms-conditions

4. User authentication flow

- /login
- /register
- /verify-otp
- /forgot-password
- /forgot-password2
- /forgot-password3
- /login-guardian
- /login-legacy
- /login-as

5. User protected flow

- /home
- Subscription plans/logs
- Emergency, legacy, guardian, family, and friend contacts
- Memories, digital diary, messages for loved ones
- Tax records and pet records
- Portfolio management
- Calendar reminders
- Document organization
- Password manager and passcode verification
- Final wishes, will and testament, funeral planning
- Collectables and keepsakes
- Profile, notifications, and payment details

Legacy Contact users have an additional flow: if their death certificate has not been approved, they are redirected to /upload-death-certificate before accessing the protected user modules.

Getting Started

Prerequisites

- Node.js 18 or newer is recommended.
- npm is recommended for this project because package-lock.json is present.
- A running backend API that matches the routes used by this frontend.

Installation

```
npm install
```

Run Locally

```
npm run dev
```

The Vite dev server starts the app with:

```
--base=/final-pass-down-frontend
```

Open the local URL printed by Vite in the terminal. Because the router also uses the `/final-pass-down-frontend` basename by default, make sure the local URL includes the same base path when testing deep routes.

Production Build

```
npm run build
```

The production build is generated in:

```
dist/
```

Preview Production Build

```
npm run preview
```

Environment Variables

The app uses Vite, but the configuration maps loaded environment variables into `process.env` through `vite.config.js`. Because of that, existing `REACT_APP_*` variables are still used in the codebase.

Create or update `.env` in the project root:

```
REACT_APP_BASE_URL=https://your-api-domain.com/api
REACT_APP_BASE_NAME=/final-pass-down-frontend
REACT_APP_WEBSITE_NAME=Final Passdown
VITE_STRIPE_PUBLISHABLE_KEY=pk_test_your_stripe_publishable_key
```

Variable Reference

Variable	Used For
REACT_APP_BASE_URL	Axios base URL in <code>src/Config/ApiConfig.jsx</code> .

Variable	Used For
REACT_APP_BASE_NAME	React Router basename and page title/path helpers.
REACT_APP_WEBSITE_NAME	Browser title formatting in helper utilities.
VITE_STRIPE_PUBLISHABLE_KEY	Stripe Elements/payment modal setup.

External Keys

index.html currently loads Google Maps through a script tag. For production, keep API keys restricted in Google Cloud Console and avoid committing private or unrestricted keys.

Available Scripts

```
npm run dev
```

Starts the Vite development server using the configured base path.

```
npm run build
```

Builds the app for production into dist/.

```
npm run preview
```

Serves the production build locally for verification.

```
npm run lint
```

Runs ESLint against src with unused-disable checks and zero warnings allowed.

Application Architecture

Entry Point

- src/main.jsx mounts the React application.
- src/App.jsx imports global CSS and renders Routers.
- src/Routers/index.jsx contains the main route map.

Global Providers

src/main.jsx wraps the app with:

- Provider from react-redux
- PersistGate from redux-persist

- `BrowserRouter`
- `UploadProgressProvider`
- `ApiConfig`

Layouts

Admin screens use:

- `src/Components/AdminLayout/Auth`
- `src/Components/AdminLayout/DashboardLayout`
- `src/Components/AdminLayout/Header`
- `src/Components/AdminLayout/Sidebar`

User screens use:

- `src/Components/UserLayout/Auth`
- `src/Components/UserLayout/UserLayout`
- `src/Components/UserLayout/Header`
- `src/Components/UserLayout/Sidebar`

Most pages import their layout directly instead of using nested layout routes.

Routing Flow

Main routing is defined in `src/Routers/index.jsx`.

Admin Routes

Admin access starts at `/admin` or `/admin/login`. After login, the admin is routed into the dashboard and management modules. Protected admin routes are wrapped with `ProtectedRoutes` and require the persisted role to match `admin`.

Important admin modules:

- Dashboard
- Profile and change password
- User management
- Legacy request management
- Legacy and guardian account details
- Subscription management
- Storage subscription management
- Storage subscription logs

- Query management
- Payment logs
- Notifications

User Routes

The user side includes public pages, auth pages, and protected account pages. Protected user routes are wrapped with `UserProtectedRoutes`, which allows:

- user
- Guardian Contact
- Legacy Contact
- Emergency Contact

After login, contact-role users may be sent to `/login-as` to choose or activate the correct role context.

Route Guards

- `GuestRoutes` redirects authenticated admins to `/admin/dashboard` and verified users away from guest auth screens.
- `ProtectedRoutes` checks admin/user role access for protected admin routes.
- `UserProtectedRoutes` checks user/contact-role access and the `localStorage.isVerified` flag.
- `HasActiveRoleGuard` prevents access to `/login-as` when a meaningful active role already exists.
- `HomeRouteGuard` redirects Legacy Contact users to death certificate upload when needed.
- `HasUploadedDetachCertificateGourd` blocks protected user modules until a Legacy Contact death certificate is approved.

Authentication And Authorization

Authentication state is stored in Redux and persisted through `redux-persist`. The main auth values are:

- token
- role
- user

These values are accessed through `src/Hooks/useAuth.jsx`.

Admin Login

Admin login uses the shared login hook in `src/Services/Auth.jsx`. On success, it stores:

- API token
- User object
- User role/type

User Login

User login handles normal users and contact-role users. The flow includes:

- API login request
- Redux token/user/role storage
- optional remember-me behavior
- OTP or passcode verification state through local storage
- contact-role routing through `/login-as`
- Legacy Contact death certificate checks

Logout

Logout clears auth-related Redux state and removes local verification values such as:

- `_code`
- `isVerified`

API Flow

Global Axios behavior is configured in `src/Config/ApiConfig.jsx`.

Base URL

```
axios.defaults.baseURL = process.env.REACT_APP_BASE_URL;
```

Request Interceptor

Each request can be enriched with:

- `Authorization: Bearer <token>`
- `timezone`

The interceptor also prefixes request paths depending on the current URL:

- Admin URLs receive /admin
- Sub-admin URLs receive /sub-admin
- Other URLs receive /

For example, an admin call like:

```
getAll("/storage-subscription/plans");
```

is sent through Axios with the admin prefix while the user is in the admin area.

Response Interceptor

When a non-login request returns 401, the app stores the intended route in `localStorage.nextTo` and redirects the user to `/verify-passcode`.

API Helpers

Shared helpers live in `src/Services/Api.jsx`:

- `getAll(route, params)`
- `getDetails(route)`
- `post(route, params)`
- `deleteData(route, params)`
- `logout(route)`

`post` and `deleteData` build `FormData` using `buildFormData` from `src/Utils/helper.jsx`.

State Management

Redux is configured in:

- `src/Store/store.jsx`
- `src/Store/reducers/rootReducer.jsx`
- `src/Store/actions.jsx`

Persisted reducer slices include:

- `data` for the current user data
- `token` for the auth token
- `role` for the active role
- `image` for image/session image state
- `reset` for reset-related state

- `logout` for logout-related state

The persisted store uses browser storage through `redux-persist/lib/storage`.

Main Feature Areas

Admin Portal

Admin pages are under `src/Pages/Admin`.

- **Dashboard:** admin overview and stats.
- **User Management:** users, legacy accounts, guardian accounts, and legacy requests.
- **Subscription Management:** platform subscription plans.
- **Storage Subscription Management:** storage plans, create/edit/detail, and active/inactive status handling.
- **Storage Subscription Logs:** records for storage subscription usage/payment history.
- **Query Management:** user contact/query submissions.
- **Payment Logs:** payment records.
- **Notifications:** admin notifications.
- **Profile:** profile update and password change.

User Portal

User pages are under `src/Pages/User`.

- **Home:** main dashboard after login.
- **Contacts:** emergency, legacy, guardian, family, and friend contacts.
- **Memories:** memories, memory messages, media uploads, and messages for loved ones.
- **Digital Diary:** personal diary entries.
- **Tax Records:** stored tax-related records.
- **Pet Records:** pet information and records.
- **Portfolio Management:** personal, retirement, and business accounts.
- **Calendar Reminder:** anniversary, birthday, holidays, new year, party events, and wedding reminders.

- **Document Organization:** day care, insurances, kids activities, job information, real estate, warranties, secrets, IDs, goals, utilities, licenses, vehicles, miscellaneous documents, and related folders.
- **Password And Passcode:** password manager, create passcode, verify passcode, and forgot passcode flow.
- **Final Wish:** final wishes, will and testament, and funeral planning.
- **Collectables And Keepsakes:** collectable and keepsake records.
- **Subscription:** plans, logs, and payment details.
- **Profile:** profile details, edit profile, and change password.
- **Legal/Public:** privacy policy, terms and conditions, and contact us.

Folder Structure

src/	
Assets/	Static asset exports
Components/	Shared reusable UI and layout components
Config/	Table headers, statuses, validations, options, API config
Hooks/	Shared React hooks
Pages/	
Admin/	Admin portal pages
User/	User portal pages
Routers/	Route definitions and route guards
Services/	API and auth service helpers
Store/	Redux store, reducers, and actions
Utils/	Helpers for form data, dates, links, files, and formatting
App.jsx	App shell
main.jsx	React entry point

Reusable Components

Common UI and helper components include:

- CustomInput
- CustomButton
- CustomModal
- CustomTable
- CustomFilters
- CustomPagination
- Toast
- Loader
- StripePaymentModal
- UploadProgressProvider
- UploadProgressModal
- UploadAndDisplayFiles
- StatsCard

- Graph
- NotificationCard
- HeaderNotification
- PhoneInput
- VideoRecorder
- MediaRecorder

Development Notes

- Prefer `npm` for installs to keep `package-lock.json` as the source of dependency resolution.
- Do not mix `npm install` and `yarn install` unless the team intentionally standardizes on Yarn.
- The app currently uses Vite with CRA-style `REACT_APP_*` variables via `vite.config.js`.
- Keep route additions centralized in `src/Routers/index.jsx`.
- Keep authenticated API calls behind the shared Axios configuration so tokens and timezone headers are applied consistently.
- When adding form submissions with files or nested objects, prefer `buildFormData` from `src/Utils/helper.jsx`.
- When adding admin tables, reuse `CustomTable`, `UseTableControls`, table headers, and table status config where possible.
- When adding protected pages, confirm whether the page should be available to only user, or also to Guardian Contact, Legacy Contact, and Emergency Contact.
- For production deployment under a different base path, update both the Vite base script/configuration and `REACT_APP_BASE_NAME`.

Deployment Checklist

Before deployment:

1. Set production values in `.env`.
2. Confirm `REACT_APP_BASE_URL` points to the production backend API.
3. Confirm `REACT_APP_BASE_NAME` matches the deployed subdirectory or `/` for root deployment.
4. Restrict Google Maps and Stripe keys to the correct domains.
5. Run linting.
6. Build the app.
7. Serve the generated `dist/` folder from the configured base path.

```
npm run lint  
npm run build
```